Log for Naive Bayes

Student id: C00313459

Name: Amisha Das

1. Introduction

This notebook explores the use of Naive Bayes machine learning model and its 2 subtypes, namely:

1. Gaussian Naive Bayes
2. Multinomial Naive Bayes

2. Gaussian Naive Bayes (GNB)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler,
KBinsDiscretizer
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.decomposition import PCA

df = pd.read_csv('diabetes.csv')
```

About Dataset-

This is a dataset about a patient's vitals and then a binary column where 0 signifies no diabetes and 1 signifies diabetes positive.

```
print("Dataset Info:\n", df.info())
print("\nFirst Few Rows:\n", df.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 995 entries, 0 to 994
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   glucose        995 non-null    int64
 1   bloodpressure  995 non-null    int64
 2   diabetes       995 non-null    int64
```

```
dtypes: int64(3)
memory usage: 23.4 KB
Dataset Info:
 None

First Few Rows:
    glucose  bloodpressure  diabetes
0        40             85         0
1        40             92         0
2        45             63         1
3        45             80         0
4        40             73         1

df = df.dropna()
```

Encoding labels to have numbers consumable by the ML model

```
label_encoders = {}
for column in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
```

Allocating columns

```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

Standardising features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Training the model.

20% reserved for test

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)
```

Selecting Gaussian Naive Bayes and fitting data into the model.

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

GaussianNB()

y_pred = nb_model.predict(X_test)
```

Results of testing

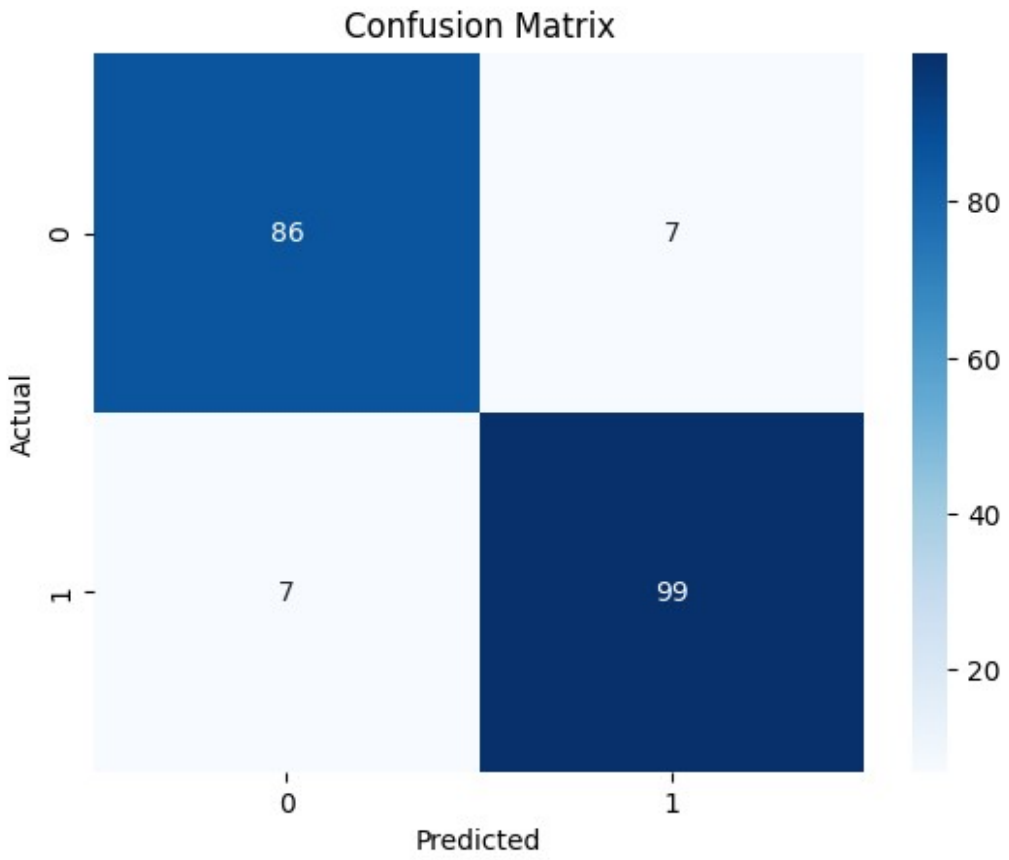
```
accuracy = accuracy_score(y_test, y_pred)
print("\nModel Accuracy:", accuracy)
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
```

```
Model Accuracy: 0.9296482412060302

Classification Report:
               precision    recall  f1-score   support

           0       0.92      0.92      0.92        93
           1       0.93      0.93      0.93       106

    accuracy                           0.93       199
   macro avg       0.93      0.93      0.93       199
weighted avg       0.93      0.93      0.93       199
```

Model achieved 92% accuracy

Confusion matrix:

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



Conclusion:

Results are excellent with 92% accuracy and confusion matrix with only 7 incorrect values.

2.1 Improving GNB Model Accuracy via PCA to reduce dimensionality

```
print("\nApplying PCA for Dimensionality Reduction:")
pca = PCA(n_components=0.95)  #reataning 95% variance
X_pca = pca.fit_transform(X_scaled)
```

```
Applying PCA for Dimensionality Reduction:
```

```
X_train_pca, X_test_pca, y_train, y_test = train_test_split(X_pca, y,
test_size=0.2, random_state=42)

nb_model_pca = GaussianNB()
nb_model_pca.fit(X_train_pca, y_train)
```

```
GaussianNB()
```

```
y_pred_pca = nb_model_pca.predict(X_test_pca)

pca_accuracy = accuracy_score(y_test, y_pred_pca)
print("\nModel Accuracy after PCA:", pca_accuracy)
```

```
print("\nClassification Report after PCA:\n",
classification_report(y_test, y_pred_pca))
```

```
Model Accuracy after PCA: 0.9296482412060302

Classification Report after PCA:
               precision    recall  f1-score   support

           0       0.93      0.91      0.92        93
           1       0.93      0.94      0.93       106

    accuracy                           0.93       199
   macro avg       0.93      0.93      0.93       199
weighted avg       0.93      0.93      0.93       199
```

Conclusion:

Unimpressive results with close to no changes. Future work required to increase accuracy from 92%.

3. Multinomial NB

```
magic_df = pd.read_csv('magic.csv')
```

About Dataset-

Major Atmospheric Gamma Imaging Cherenkov (MAGIC) telescope binary classification set to classify an event as gamma ray signal (g) or background noise from hardons (h)

Since the data was converted from a .data file, need to add features names.

```
feature_names = ["fLength", "fWidth", "fSize", "fConc", "fConc1",
"fAsym", "fM3Long", "fM3Trans", "fAlpha", "fDist", "class"]
magic_df.columns = feature_names

magic_df['class'] = magic_df['class'].map({'g': 1, 'h': 0})
```

Feature decretizisation via bins, diving into 10 discrete bins. Discreet values are stored in X_nagic_binned

```
kbins = KBinsDiscretizer(n_bins=10, encode='ordinal',
strategy='uniform')
X_magic = magic_df.iloc[:, :-1]
y_magic = magic_df['class']
X_magic_binned = kbins.fit_transform(X_magic)
```

Splitting data into train and test sets (20-80)

```
X_train_magic, X_test_magic, y_train_magic, y_test_magic =
train_test_split(X_magic_binned, y_magic, test_size=0.2,
random_state=42)
```

Feeding sets into the Multinomial Naive Bayes Model.

```
multi_nb = MultinomialNB()
multi_nb.fit(X_train_magic, y_train_magic)
```

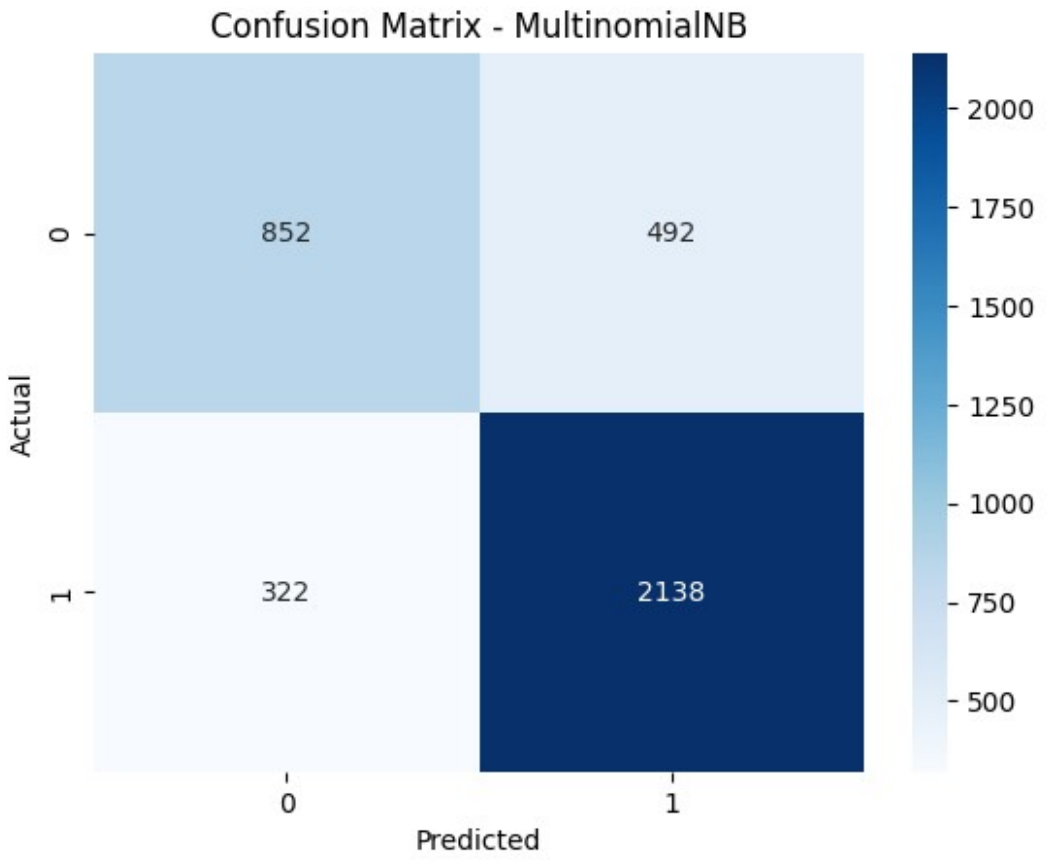
```
MultinomialNB()
```

Making Predictions

```
y_pred_magic = multi_nb.predict(X_test_magic)

multi_accuracy = accuracy_score(y_test_magic, y_pred_magic)
print("\nMultinomialNB Model Accuracy:", multi_accuracy)
print("\nMultinomialNB Classification Report:\n",
classification_report(y_test_magic, y_pred_magic))
```

```
MultinomialNB Model Accuracy: 0.7860147213459516

MultinomialNB Classification Report:
               precision    recall  f1-score   support

           0       0.73      0.63      0.68      1344
           1       0.81      0.87      0.84      2460

    accuracy                           0.79      3804
   macro avg       0.77      0.75      0.76      3804
weighted avg       0.78      0.79      0.78      3804
```

```
cm_magic = confusion_matrix(y_test_magic, y_pred_magic)
sns.heatmap(cm_magic, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - MultinomialNB')
plt.show()
```



Conclusion:

Results are unimpressive at 78% accuracy and low recall and f1 scores. Further work required.

4. Log:

```
log = pd.read_csv("log_table.csv")
pd.set_option('display.max_colwidth', None)
log
```

```
                      Section  \
0  Gaussian Naive Bayes (GNB)
1  Gaussian Naive Bayes (GNB)
2  Gaussian Naive Bayes (GNB)
3               GNB with PCA
4               GNB with PCA
5               GNB with PCA
6             Multinomial NB
7             Multinomial NB
8             Multinomial NB
9             Multinomial NB

                                                 Description  \
```

```
0                           Added count-based diabetes data
1  Added LabelEncoder() to convert categories into numbers
2                Fixed column assignment issue in magic_df
3                 Tried increasing model accuracy with PCA
4                        Removed redundant fit_transform()
5                Results negligible, noted for future work
6      Added MultinomialNB and KBinsDiscretizer to imports
7       Tried working with .data file but it wouldn't work
8                Added error checking for DataFrame format
9    Converted .data file to .csv for better compatibility

  Level of Difficulty Time Spent (approx.)  \
0              Medium               30 min
1                Easy               20 min
2              Medium               40 min
3                Hard               1 hour
4                Easy               15 min
5                 NaN                  NaN
6                Easy               15 min
7              Medium               30 min
8                Easy               20 min
9              Medium               40 min

                                    Change From  \
0                                           NaN
1                                           NaN
2  \nmagic_df.columns = feature_names\n
3                                           NaN
4               Redundant fit_transform() usage
5                                           NaN
6                                           NaN
7                                           NaN
8  \nmagic_df.columns = feature_names\n
9                             .data file format


Changed To
0  import pandas as pd\nimport numpy as np\nimport matplotlib.pyplot
as plt\nimport seaborn as sns\nfrom sklearn.model_selection import
train_test_split\nfrom sklearn.preprocessing import LabelEncoder,
StandardScaler, KBinsDiscretizer\nfrom sklearn.naive_bayes import
GaussianNB, MultinomialNB\nfrom sklearn.metrics import accuracy_score,
classification_report, confusion_matrix\nfrom sklearn.decomposition
import PCA\n
1
df = pd.read_csv('diabetes.csv')\n
2
\nif isinstance(magic_df, pd.DataFrame):\n    magic_df.columns =
feature_names\nelse:\n    print("Error: magic_df is not a DataFrame.
Check file reading.")\n
```

```
3
print("Dataset Info:\n", df.info())\nprint("\nFirst Few Rows:\n",
df.head())\n
4
Removed fit_transform() where unnecessary
5
NaN
6
df = df.dropna()\n
7
Encountered file compatibility issues
8
\nif isinstance(magic_df, pd.DataFrame):\n    magic_df.columns =
feature_names\nelse:\n    print("Error: magic_df is not a DataFrame.
Check file reading.")\n
9
Converted .data file to .csv using Pandas
```

References, Acknowledgment and Appendices

- https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Chatgpt for findign data set, ideas and debugging
- https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope